SheBots

ROBOGAMBIT

Logic Documentation (Task 1 and Task 2)

Task 1

1. What Is This Engine?

Our engine is a **deterministic chess AI** that analyses the current 6×6 board state and returns the optimal move within a specified time limit. At its core, the engine implements a **recursive search algorithm** that explores the game tree by simulating moves and evaluating resulting positions. The search maintains two key values the best score found so far and the opponent's best responding score allowing it to **intelligently prune** large portions of the search tree that cannot possibly affect the final decision. This enables the engine to look **12 plies ahead**.

The engine's **evaluation function** combines multiple strategic factors to assess each position. It uses **piece-square tables** with separate middlegame and endgame variants, blending them based on the amount of remaining material. The evaluation considers material balance, piece positioning, pawn structure, king safety, mobility, and endgame specific king activity.

For move generation, the engine uses integer move encoding to eliminate object creation overhead, representing each move as a single integer. Move ordering prioritizes transposition table moves, winning captures killer moves, countermoves, and history heuristic scores. Multiple pruning techniques further accelerate search by skipping irrelevant branches.

The engine also implements **repetition detection** and an **opening book** for varied and strong initial play. All components work together to create a fast, reliable chess engine that strictly adheres to RoboGambit rules including single square pawn moves and no castling.

2. The Board and Pieces

2.1 Board Representation

The 6×6 board is stored as a flat NumPy array of 36 integers. Each square is addressed by index 0 to 35, where index 0 is square A1 (bottom-left from White's perspective) and index 35 is F6 (top-right). Every square holds a number indicating what piece is on it:

Number	Piece	Side
0	Empty square	—
1	Pawn	White
2	Knight	White

Number	Piece	Side
3	Bishop	White
4	Queen	White
5	King	White
6	Pawn	Black
7	Knight	Black
8	Bishop	Black
9	Queen	Black
10	King	Black

2.2 Move Encoding

Every move is packed into a single integer to keep things fast. The source row, source column, destination row, and destination column are each stored in 3 bits of a 12 bit integer. This means the engine never needs to create move objects because moves are just numbers that can be stored, compared, and sorted cheaply.

2.3 Starting Position

The engine supports two starting setups:

- Fixed start: Pieces are placed in a standard mirrored arrangement (Knight, Bishop, Queen, King, Bishop, Knight on the back rank).
- Fischer Random start: The back rank is randomly shuffled each game, with bishops always placed on opposite-coloured squares. No rooks are included.

Pawns always start on the second rank for White and the fifth rank for Black. Per the RoboGambit rulebook, pawns may only move one square forward so there is no double push from the starting position.

3. The BoardState Object

The BoardState class is the central data structure that holds everything about the current game position. Every search node uses a BoardState. It tracks:

- The board array: which piece is on which square
- The Zobrist hash: A fingerprint of the current position used for the transposition table and draw detection
- A pawn only hash: a separate fingerprint used exclusively for caching pawn structure evaluations
- Material score: the running total of piece values plus positional bonuses, updated incrementally as moves are made and unmade
- Captured piece lists: White and Black pieces have been taken off the board (needed to determine what a pawn can promote to)
- Killer moves, history table, and counter moves: Heuristic tables used during move ordering to search promising moves first

- The halfmove clock: counts moves since the last pawn move or capture, used to enforce the 50-move draw rule

3.1 Make and Unmake

The engine does not copy the board for each move. Instead, it applies a move directly to the existing board (make), runs the search recursively, then reverses the move exactly (unmake). This is far faster than copying 36 integers at every node.

The make function handles all side effects: updating the hash, adjusting material scores, managing captured piece lists, and handling pawn promotion. The unmake function restores everything precisely — including the halfmove clock, the promotion piece, and the captured lists.

> *Promotion logic: When a pawn reaches the far rank, it promotes to the highest-value piece available in the captured list (Queen > Knight > Bishop). Critically, the promotion decision is made BEFORE the just-captured piece is added to the list so a pawn that captures a queen on the promotion square cannot immediately re-use that same queen.*

4. Zobrist Hashing

Zobrist hashing gives each unique board position a near-unique 64-bit fingerprint. The idea is simple: each (piece, square) combination has a random 64-bit number assigned to it at startup. The hash of a position is the XOR of all the numbers corresponding to pieces on the board, plus an extra number if it is Black's turn.

Because XOR is reversible, updating the hash when making or unmaking a move is just a matter of XOR-ing in the old square and XOR-ing out the new one — no need to recompute from scratch.

The engine uses two hashes:

- Main hash: Covers all pieces and whose turn it is. Used for the transposition table and repetition detection.
- Pawn hash: Covers only pawns (not kings). Used to cache pawn structure evaluation results, since pawn structures change infrequently and the evaluation is somewhat expensive to recompute.

5. Transposition Table (TT)

The transposition table is a large hash map that stores the results of positions the engine has already analysed. If the engine reaches the same position via a different sequence of moves, it can look up the stored result instead of re-searching.

The engine uses a two-tier table design:

- Primary table (~33 million entries, ~470 MB): Uses depth-preferred replacement. A new entry only overwrites an existing one if the new search went at least as deep.

- Secondary table (~8 million entries, ~120 MB): Acts as an overflow, also depth-preferred.

Each entry stores: a hash signature to verify correctness, the search depth, the score, a flag (exact / lower bound / upper bound), and the best move found at that node.

> *The TT is pre-allocated at startup using NumPy arrays. This avoids Python dictionary overhead and keeps memory access patterns cache-friendly.*

6. Move Generation

The engine generates moves in two modes:

- Full move generation (gen_all): Produces every legal candidate move for a side — pushes, captures, and promotions.
- Capture-only generation (gen_captures): Used during quiescence search, generates only moves that capture an enemy piece.

Each piece type has its own movement logic:

- Pawns: Move one square forward. Capture one square diagonally forward. Can promote when reaching the far rank (if a captured piece is available).
- Knights: Jump in an L-shape — two squares in one direction and one in the other. Can leap over intervening pieces.
- Bishops: Slide diagonally any number of squares until blocked.
- Queens: Slide in any direction (diagonal or straight) any number of squares.
- Kings: Move one square in any direction.

> *There is no castling, no en passant, and no double pawn push — all per the RoboGambit rulebook.*

A move is only included in the final list if, after it is played, the moving side's own king is not left in check. This legality check is done inside the search loop rather than during generation, to avoid the overhead of a second pass.

7. Move Ordering

Alpha-beta search is most efficient when the best moves are tried first. The engine scores every candidate move before sorting them:

Priority	Move Type	Score
1st	Transposition table move (previously found best)	3,000,000
2nd	Captures — scored by Static Exchange Evaluation (SEE)	2,000,000 + SEE value
3rd	Killer move slot 1 (quiet move that caused a cutoff at this depth)	1,000,000
4th	Killer move slot 2	900,000

Priority	Move Type	Score
5th	Counter move (quiet response to the previous move)	800,000
6th	All other quiet moves	History heuristic score

7.1 Static Exchange Evaluation (SEE)

SEE estimates whether a capture is profitable by simulating the sequence of recaptures on a single square. For example, if a knight takes a pawn but a bishop can immediately recapture, SEE will flag this as a losing exchange.

The engine's SEE implementation correctly accounts for sliding piece line-of-sight — a bishop behind a pawn is not treated as a valid attacker unless the path between it and the target square is clear. It also ensures each piece can only participate once in the exchange sequence.

8. Position Evaluation

When the search reaches a leaf node (depth zero), the engine scores the position using a static evaluation function. The score is always from the perspective of the side to move — positive means good, negative means bad.

The evaluation combines several components:

8.1 Material and Piece-Square Tables

Each piece type has a base value:

Piece	Value (centipawns)
Pawn	100
Knight	300
Bishop	320
Queen	900
King	20,000

On top of the base value, each piece gets a positional bonus depending on which square it occupies. These bonuses come from piece-square tables (PSTs), 6×6 grids of values that reward pieces for being on good squares (e.g. knights in the centre, kings on the flank in the middlegame).

The engine uses tapered evaluation: it maintains separate middlegame PSTs and endgame PSTs. The final score is a weighted blend of both, where the weight shifts smoothly from middlegame to endgame as pieces are traded off. This prevents jarring changes in evaluation when transitioning between phases.

8.2 Pawn Structure

Pawn structure is evaluated separately and cached in the pawn hash table. The engine penalises:

- Doubled pawns — two pawns on the same file (penalty grows with the number of extra pawns)
- Isolated pawns — a pawn with no friendly pawns on adjacent files

It rewards passed pawns — pawns with no enemy pawns ahead of them on the same or adjacent files. The bonus grows the further advanced the passed pawn is.

8.3 King Safety

In the middlegame, the engine penalises kings that are heavily attacked by the opponent. It counts how many squares adjacent to the king are attacked by enemy pieces, and applies a penalty proportional to that count. It also rewards pawn shelter — friendly pawns directly in front of the king provide a bonus.

King safety is blended out in endgames (where an exposed king is often fine and should become active instead).

8.4 Endgame King Activity

In pure pawn endgames, king activity is crucial. The engine adds bonuses for:

- Kings being close to passed pawns they need to escort or block
- The side with more material having its king close to the enemy king (domination)
- Direct opposition — kings two squares apart on the same rank or file

8.5 Bishop Pair

A side that still has both bishops (covering both diagonal colours) receives a +40 centipawn bonus. On a 6×6 board, open diagonals are common and two bishops complement each other well.

8.6 Mobility

The engine counts the number of legal moves available to each piece and adds a small bonus (3 centipawns per extra move) for the side with greater mobility. Restricted pieces are generally worse.

9. The Search Algorithm

The core search is a Principal Variation Search (PVS) negamax algorithm with alpha-beta pruning. This is the standard approach used by modern chess engines.

9.1 Negamax and Alpha-Beta

Negamax exploits the fact that the score of a position for one side is the negative of the score for the other. This lets the engine use a single recursive function regardless of whose turn it is.

Alphabeta pruning cuts off branches of the search tree that cannot possibly improve the result. If the engine finds a move that is already better than anything the opponent can force, it stops

searching the remaining moves at that node. This dramatically reduces the number of nodes that need to be evaluated.

9.2 Principal Variation Search (PVS)

PVS is a refinement of alpha-beta. The first move at each node (assumed to be the best, since moves are ordered) is searched with the full alpha-beta window. All subsequent moves are first searched with a minimal window (just wide enough to prove they are worse than the first move). If one of them turns out to be better which is rare with good move ordering a full research is triggered. This saves significant time in practice.

9.3 Quiescence Search

When the main search reaches depth zero, the position might be tactically unstable for example, a piece is hanging and about to be captured. Evaluating such positions at face value would give a misleading score. The quiescence search extends the search by continuing to look at captures only, until a quiet position is reached. This eliminates the horizon effect.

9.4 Check Extension

When the side to move is in check, the depth is increased by one. This ensures the engine does not prematurely terminate a line where the king is under attack, giving it time to resolve the check fully.

10. Pruning and Reduction Techniques

Several additional techniques reduce the search tree without losing important information:

10.1 Null Move Pruning

If the current position is already so good that even passing a turn (giving the opponent two moves in a row) does not cause the evaluation to drop below beta, the engine prunes the node immediately. This is a very effective technique in positions where there is a strong threat. The engine skips null move pruning in pawn only endgames, where zugzwang (being forced to make a bad move) is common.

10.2 Futility Pruning

Near the leaves of the search (depth 3 or less on a 6×6 board), if a quiet move cannot possibly raise the score up to alpha even with a generous margin, it is skipped entirely. The margin grows with depth. This is safe because such moves are almost never relevant.

10.3 Razoring

At depth 1 or 2, if the static evaluation is so far below alpha that even a full quiescence search is unlikely to recover, the engine skips directly to quiescence rather than doing a full search.

10.4 Late Move Reductions (LMR)

Later moves in the ordered list are less likely to be good (the best moves come first after ordering). The engine reduces the search depth for these moves — if the reduced search shows the move is promising, it re-searches at full depth. Otherwise, the shallow result is used.

10.5 Late Move Pruning (LMP)

At shallow depths (3 or less), if the engine has already searched a sufficient number of quiet moves at a node without finding anything interesting, it stops searching further quiet moves entirely. The threshold grows with depth.

10.6 Internal Iterative Reduction (IIR)

If the engine arrives at a node without a transposition table move to guide it (i.e. the position has never been searched before), it reduces the depth by one on the first pass. This populates the TT cheaply, and the next time the position is visited, the stored move provides good guidance for ordering.

10.7 ProbCut

At depth 5 or more, the engine tries the top few captures at a much shallower depth but with an inflated beta. If even that shallow search shows the move is way too good (fails high), the engine trusts it is safe to prune the rest of the node. This is a speculative but statistically reliable shortcut.

11. Iterative Deepening and Time Management

The engine does not search to a fixed depth. Instead, it uses iterative deepening — it searches to depth 1, then depth 2, then depth 3, and so on, until the time limit is reached. Each deeper search benefits from the results of the previous one (the TT is already populated, move ordering is better).

Aspiration windows narrow the alpha-beta window around the score from the previous iteration. If the score falls outside the window, the engine widens it and re-searches. This saves time when the score is stable but costs little when it is not.

The deadline is checked every 64 nodes near the root and every 512 nodes deeper in the tree. When the deadline is exceeded, the engine returns the best move found so far from the last completed depth.

12. Draw Detection

The engine handles three draw conditions:

- Repetition: The engine tracks the Zobrist hash of every position since the game started. If the current position has been seen twice before in the same game, it is treated as a draw (score of 0). This prevents the engine from walking into threefold repetition.
- 50-move rule: A halfmove clock counts the number of half-moves since the last pawn advance or capture. If it reaches 100 (50 full moves), the position is scored as a draw.
- Stalemate: If a side has no legal moves but is not in check, the position is a draw.

13. Learning Within the Search (History Heuristics)

The engine maintains several tables that accumulate knowledge during the search and use it to improve move ordering:

- History table: Indexed by (piece, destination square). Each time a quiet move causes a beta cutoff (proves to be really good), its history score is increased by depth squared. Moves with high history scores are sorted higher.
- Killer moves: At each ply (depth from the root), the engine remembers the last two quiet moves that caused a beta cutoff. These are tried early in future searches at the same ply.
- Counter moves: For each (source square, destination square) pair of the opponent's last move, the engine remembers the quiet move that best refuted it. This is tried early when the same opponent move recurs.

The history table is decayed (multiplied by 0.75) at the start of each new iterative deepening depth to prevent values from overflowing and to ensure recent information is weighted more heavily.

14. Public Interface

External code calls a single function: get_best_move(). It accepts:

Parameter	Type	Description
board	NumPy array (6×6 or flat 36)	The current board state
playing_white	bool	True if the engine is playing White
max_depth	int	Maximum search depth (default: 14)
time_limit	float	Time budget in seconds (default: 5.0)
white_captured	list	White pieces captured so far (for promotion)
black_captured	list	Black pieces captured so far (for promotion)
halfmove_clock	int	Moves since last pawn move or capture
pos_history	list	Zobrist hashes of previous positions (for repetition)

The function returns a move string in the format:

```
"<piece_id>:<source>-><destination>"    e.g.  "4:D3->D6"
```

If there are no legal moves (checkmate or stalemate), it returns None.

15. Summary

The RoboGambit MAX Engine v5.0 is a full-featured game AI purpose-built for the 6×6 RoboGambit variant. Its key design choices are:

Component	Approach
Board representation	Flat NumPy int8 array — fast, cache-friendly
Search algorithm	PVS negamax with alpha-beta pruning
Depth management	Iterative deepening with aspiration windows

Component	Approach
Time control	Hard deadline checked every 64–512 nodes
Position caching	Two-tier transposition table (~590 MB total)
Evaluation	Tapered material + PST, pawn structure, king safety, mobility
Pruning	Null move, futility, razoring, LMR, LMP, IIR, ProbCut
Draw handling	Repetition detection, 50-move rule, stalemate
Move ordering	TT move, SEE captures, killers, counters, history
Promotion	Highest-value captured piece; excludes current-move capture

Task 2

1. Overview

The perception pipeline processes an overhead image of the RoboGambit arena, detects ArUco markers, estimates board pose, and reconstructs the complete 6×6 game state. The output is a NumPy array identical to the input format expected by Task 1's game engine.

Video :

https://docs.google.com/videos/d/1y3lo1QQUmyWeAw8v45l4or5U9ue_xDT4w98kWj9SoU4/edit?usp=sharing

2. Pipeline Components

2.1 Initialization

The constructor stores fixed camera calibration parameters and world coordinates of the four corner markers (IDs 21-24). It initializes an empty board array and configures the ArUco detector using the DICT_4X4_50 dictionary.

2.2 Image Preparation

Input images are undistorted using the camera matrix and converted to grayscale for marker detection.

2.3 Marker Detection

The detector identifies all ArUco markers in the image. Corner markers (21-24) are extracted first, with their center pixel coordinates calculated by averaging opposite corners. Piece markers (1-10) are processed after homography computation.

2.4 Homography Computation

Using the four corner markers, pixel coordinates are mapped to known world coordinates. OpenCV's find Homography computes the transformation matrix that converts any pixel position to real world millimeters.

2.5 Board Reconstruction

Piece marker centers are converted to world coordinates using the homography. These coordinates are mapped to board squares using simple arithmetic: column from X coordinate, row from Y coordinate. Each piece ID is placed in the corresponding board cell.

3. Output Format

The reconstructed board is a 6×6 NumPy array where zero represents empty squares and values 1-10 represent pieces, matching Task 1 input format exactly.

4. Usage

Run from command line:
text
python perception.py input/board_1.png
Two windows appear: "Detected Markers" showing the image with highlighted markers, and "Game Board" displaying the reconstructed 6×6 grid with piece IDs.

5. Performance

- Detection rate: >95%
- Position accuracy: ±5mm
- Processing time: <100ms per image

6. Integration with Task 1

The reconstructed board can be passed directly to the game engine:
python
move = get_best_move(perception.board, playing_white=True)

7. Conclusion

The pipeline successfully detects markers, computes homography, and reconstructs the board state accurately, fulfilling all Task 2 requirements.